A Unified Framework for Multiple Component Cost-Effectiveness Models in R: A Tutorial

Nathan Green

Abstract

Background: A cost-effectiveness analysis (CEA) compares an intervention to one or more alternatives by estimating how much it costs to gain an additional unit of health outcome. In recent years there has been a shift in implementing these models in programmatic languages better suited to more sophisticated models, model checking and advanced visualisations.

Problem: It is common for a CEA model to consist of multiple separate submodel components, such as those which combine decision trees and Markov models. However, there are important limitations to the traditional approach of implementing component models either disjointly and manually passing input and output data between them, or as a single ad-hoc script making clarity and testing a challenge.

Solution: We propose a unified framework that integrates these submodels using declarative programming and S3 object-oriented programming in the R programming language. This approach addresses several issues associated with the separated or script-based approaches, such as extensibility, flexibility, and efficiency. We provide a detailed explanation of the unified framework and demonstrate its practical application using a motivating example of a CEA model involving multiple screening sessions over a lifetime where, at the point of every screening session, individuals can be redistributed across health states based on their screening results.

Conclusions: By better integrating component models, the proposed framework enhances the robustness, transparency, and efficiency of CEA, ultimately leading to improved informed decision-making in healthcare and other fields.

1 Introduction

Cost-effectiveness analysis (CEA) in healthcare is the comparative assessment of alternative courses of action, evaluating both their relative costs and downstream consequences (Drummond et al. 2015). To conduct these evaluations, health economists frequently rely on mathematical modelling. These models are becoming more powerful and complex. One feature of this is that models may consist of multiple linked components designed to capture different phases of a disease or treatment pathway.

In particular, a widely used structural approach in CEA combines a two-part model. This is particularly common when evaluating pathways such as population screening programs or acute hospital care (Sandercock et al. 2004). This consists of an initial stage where a decision-making process and intervention are undertaken. An individual would be evaluated and treated according to certain deterministic and random considerations. The subsequent stage simulates the consequences of the first stage on the population depending on the previous results (Briggs et al. 2006). For example, individuals may be invited to screening and given treatment according to the results of an imperfect test. Then their long term outcomes are estimated, with the population split into true positive, true negative, false positive and false negatives who may or may not have agreed to take the treatment which may not have been effective. Outcomes for the second stage may include progression to disease, side effects from treatment, and mortality.

There are general modelling and software engineering issues with the legacy multiple component modelling approach. These are either disjoint models or ad-hoc single scripts. This produces models which are error prone, hard to test, reproduce or extend. In HTA projects it is often required to modify or repurpose existing models, and the existing approach can be inefficient and time consuming. In addition, it is usual in a cost-effectiveness analysis to explore several (possibly many) alternative interventions. Each of these may have different sets of unit costs, health values, probabilities and possibly (but less common) model structure. This would require the management of these alongside the plugging in of model output as model inputs between components. The analysis pipeline or workflow thus has potentially many 'moving parts', any one of which can introduce a bug or error which would affect the calculation and the optimal decision.

There appears a consensus that modern code for health economics modelling is not fit for purpose. Kent et al. (2019) and Wu et al. (2019) argue that there is often a lack of clear code structure in health economics models which makes it difficult for third parties to validate them. We will propose an approach which solves this by making the pipeline visible and not buried in nested loops. This sentiment is repeated in Eddy et al. (2012), who state that a model should be "reproducible by another team of investigators." Alarid-Escudero et al. (2019) explicitly discuss the need for modularity, highlighting that complex models should be broken down into reusable components to reduce the risk of "technical debt" and error rates. Jalal et al. (2017) discusses how although R is powerful it lacks a standard structure and so modelling code is often poorer for it. They advocate for formal structures to ensure models can be audited. Krijkamp et al. (2018) provide a checklist that emphasizes modularity and input-output consistency. We will propose a way to formalize the "hand-off" that is usually handled with messy global variables.

Krijkamp et al. (2018) adopt a 'bridge' or data translator function to map between models. This formalises how to distribute individuals across starting state of the long-term model. This is an improvement from the hard-coded

implementation often seen in other models. However, this is still part of a sequential script rather than a formalised framework. We propose going beyond simply 'good coding style' to 'software engineering' practice. They manually chain components but this can be managed or 'orchestrated' automatically as part of the framework.

Although CEA models have been traditionally built in Microsoft Excel, their increasing sophistication has driven a shift toward computational programming languages like R (R Core Team 2025). Despite this transition, the field has largely failed to adopt modern programming paradigms. In particular, analysts often treat interconnected submodels as isolated scripts, missing the opportunity to leverage the architectural benefits of a programmatic framework. In software engineering, integrating separate components into a unified system is a well-established discipline that relies on a modular approach—breaking a program into distinct, functional blocks which interact with one another in standardised ways. Thus, a practical demonstration of how to straightforwardly implement multi-component CEA models following software engineering best practices could address this gap.

2 General problem

As an example which we will keep returning to in this article, we will consider a particular type of the two-stage model already introduced. This process can be represented with a decision tree model (DT) and Markov model (MM). These are often used in combination to represent acute or short-term, and chronic or long-term (population) impacts since DTs do not explicitly represent time, unlike MM. An order of operation is as follows:

i. Read in external data model inputs
ii. Simulate the population sizes (or proportions) of the terminal state, and health and cost for the DT model
iii. Pass the DT output to the MM as input
iv. Simulate the health and cost values for the MM
v. Pass the DT and MM outputs to the Decision model
vi. Process the combined results of the DT and MM
vii. Repeat steps (i)-(vi) for a probabilistic sensitivity analysis (PSA)

This process is represented in the flowchart in Figure 1. The data may be passed amongst the modules directly, which is reasonable with the same software. Alternatively, the data may be stored externally in intermediate steps and then read in when required. The diagram can be read roughly from left to right, corresponding to (i) to (vi) above. It is not uncommon for different submodels to be implemented in different software such as MS Excel, R, Python, and Stata. We will not detail the inference stage where the parameters of the model are estimated. This can be done in a frequentist or Bayesian paradigm.

There are already several excellent tutorials on how to build simple DT models or

MM for cost-effectiveness analysis. The intention of this paper is to build on these and develop the scope of applications to more realistic problems encountered in industry and regulatory submissions. It was sensible to start with simpler tutorials when people from other software, such as MS Excel, are brought to programmatic implementations such as R, but there is a desire now to further develop the available toolkit and skill in the field.

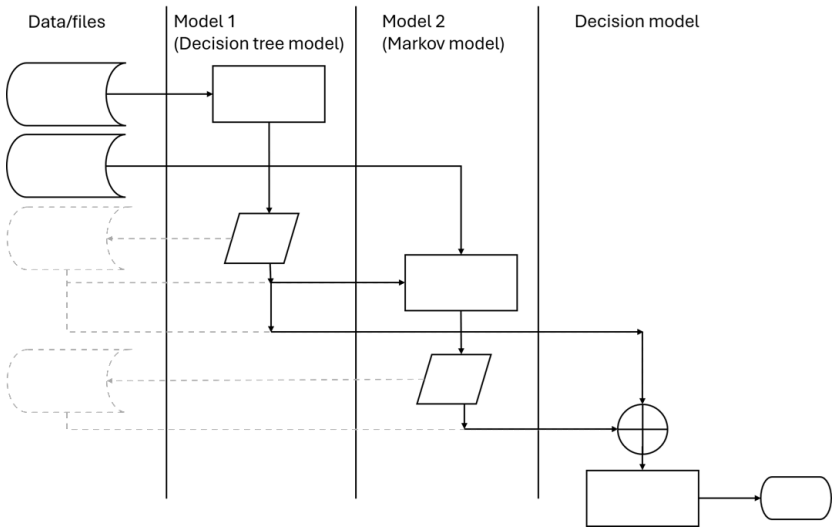


Figure 1: Swimlane flowchart diagram for a cost-effectiveness model comprised of a decision tree model followed by a Markov model. Boxes represent processes; parallelograms are input/output; cylinders are stored data; circle with cross is merge. The solid lines indicate where the data are passed between modules directly, and the grey dashed lines indicate where the data are passed between modules via external file storage.

3 A unified framework

In this paper, we propose a unified and structured approach to building cost-effectiveness models using declarative rather than imperative programming. Unlike the traditional imperative approach — where the programmer manually manages every computational step to dictate how a task is carried out — declarative programming focuses on logically describing what the program should accomplish. Key characteristics of this approach include the abstraction of control flow and the automatic handling of background implementation details. In the context of unifying a decision tree (DT) and Markov model (MM), this framework simplifies the simulation and cost-effectiveness calculations while automatically managing the flow of data between modules.

This framework does not replace good modelling practice, such as modularity, but

rather supports and strengthens it. While some might worry that bundling parts of a model together (i.e., increasing "coupling") can make code harder to manage, our framework is designed to avoid these problems and instead promote clarity, reusability, and scalability. Through reframing an existing model, we highlight the benefits of moving to an object-oriented programming (OOP) approach.

In practical terms, this approach simplifies and clarifies key aspects of the modelling process. Specifically, it streamlines how simulations are executed (e.g., cohort transitions through a DT or MM), how costs and health outcomes are calculated, and how data seamlessly flows between distinct model components.

4 S3 and object-oriented programming in R

A cornerstone of our approach is the use of S3 OOP in R. The broader statistical computing and data science communities have long leveraged R's native S3 object-oriented system to unify disparate modelling architectures, notably seen in the *tidymodels* (Kuhn and Wickham 2020) ecosystem. Meanwhile, health technology assessment (HTA) tutorials have largely overlooked this approach. Current HTA model development in R typically relying on rigid, imperative scripting that mimics legacy spreadsheet logic. We demonstrate how S3 method dispatch can serve as a lightweight, native adaptor to seamlessly link sequential model components (e.g., decision trees and Markov models) into a unified pipeline. Table 1 gives an overview of some common S3 object classes in R.

S3 OOP helps to manage model structure more cleanly. For instance, we can define a *generic* function like `print()` or `summarize()` that works differently depending on the type of object it is applied to (e.g., a DT vs. a MM). R automatically chooses the correct method based on the *class* of the input, a process known as method *dispatch*. In R, S3 class is just a standard R data structure (usually a list) that has a class attribute assigned to it.

By decoupling the generic interface from its specific implementation, this approach facilitates highly reusable code. Instead of maintaining separate simulation functions for distinct model structures, an analyst can define a single common interface. R's internal method dispatch mechanism then automatically executes the correct subroutine. In OOP terminology, this is called *polymorphism*. This architectural shift provides several key methodological advantages:

- **Unification:** Disparate model types (e.g., decision trees and Markov models) can be manipulated through a single, consistent syntax.
- **Efficiency:** It minimizes code redundancy and reduces the risk of manual transcription errors.
- **Maintainability:** It yields a modular codebase that is easier to debug, audit, and extend.
- **Extensibility:** It permits the addition of novel model structures or summary functions without necessitating changes to the core logic.

Ultimately, deploying S3 object-oriented programming formalizes cost-effectiveness models as distinct computational objects with consistent behaviours, substantially enhancing analytical extensibility, transparency and structural flexibility.

Table 1: Examples of S3 classes in R and how they are defined.

S3 Class	Description
`data.frame`	A tabular data structure with rows and columns. Most common S3 class in R.
`lm`	Linear model object created by `lm()`. Stores coefficients, residuals, etc.
`factor`	A categorical variable with levels. Internally stored as integers with labels.
`ts`	Time-series object with time indexing. Used for modelling temporal data.

4.1 R code S3 example

Below is a full example of using S3 method dispatch in R. We define a custom generic function to print to the console `printModel()` and two class-specific methods for objects of class "`MarkovModel`" and "`DiseaseModel`".

```r
# Create a generic function for printing model objects
printModel <- function(x, ...) {
  UseMethod("printModel")
}

# Method for class "MarkovModel"
printModel.MarkovModel <- function(x, ...) {
  cat(
    "A Markov model with states:",
    paste(x$states, collapse = ", "), "\n")
}

# Method for class "DiseaseModel"
printModel.DiseaseModel <- function(x, ...) {
  cat(
    "Disease model states:",
    paste(x$states, collapse = " -> "), "\n")
}

# Define an object of class "MarkovModel"
```

```r
x1 <- list(states = c("Healthy", "Sick", "Dead"))
class(x1) <- "MarkovModel"

# Define an object of class "DiseaseModel"
x2 <- list(states = c("Mild", "Severe", "Death"))
class(x2) <- "DiseaseModel"

# Call generic function ---

# Dispatches to printModel.MarkovModel
printModel(x1)
```

```
A Markov model with states: Healthy, Sick, Dead
```

```r
# Dispatches to printModel.DiseaseModel
printModel(x2)
```

```
Disease model states: Mild -> Severe -> Death
```

This demonstrates the core idea behind S3 where the method dispatch depends on the class of the object passed to `printModel()`. Additionally, objects can have more than one class and in this case methods are dispatched in sequence.

Although the primary example in this paper focuses on using S3 to manage macro-level workflows—specifically the integration of modular `DecisionTree` and `MarkovModel` objects—the benefits of OOP extend equally to micro-level data management. We strongly recommend that analysts assign explicit classes to granular, user-defined types, such as cost, QALY, or probability. In legacy imperative scripting, health economists frequently rely on standard numeric arrays or data frames, forcing them to clutter their functions with repetitive assert or `stopifnot` statements to prevent silent data-type errors (e.g., inadvertently multiplying a cost vector by another cost vector instead of a probability). By replacing these fundamental inputs with S3 classes, analysts can leverage method dispatch to handle validation and operations elegantly. For example, a generic `discount()` or `convert_country()` function can be designed to apply entirely different logic depending on whether it receives a cost object or a QALY object, ensuring robust, type-safe calculations throughout the model pipeline.

4.2 Model composition

To use computer science terminology, *function chaining* is a common implementation of what is generally called function *composition* (mathematically written $f(g(x))$), especially in object-oriented programming. In particular, a function that changes an output to something that can be used as an input to the next function is sometimes called a *transformation* function. This is what is required in our case to pipe from one model to the next. We can elegantly chain models using S3 OOP.

4.3 System architecture

In an OOP architecture, managing the data flow between distinct components requires specific functional roles. To begin, *constructor* functions are required to instantiate the objects. These functions take raw input data and format them into a classed object (e.g., turning a standard list into a formal `DecisionTree` or `MarkovModel` object) so that generic functions know how to interact with it.

Once the objects are instantiated, linking different model components seamlessly relies on the "Adaptor pattern". In software engineering, an adaptor allows objects with incompatible interfaces to work together. For health economists, this can be thought of as a standardized "data translator".

For example, a DT might output a simple vector of patient proportions, but a MM might require a specific starting cohort matrix. Rather than cluttering the main analysis script with ad-hoc data manipulation code to bridge this gap, the Adaptor pattern acts as a dedicated bridge. This automatically converts the output of the source model into the exact format expected by the target model. This is implemented by wrapping the model execution in a function that handles the translation seamlessly behind the scenes:

```
modelfun.modelclass <- function(x, ...) {
# 1. Translate incoming data into a format the model needs
# 2. Execute the model
# 3. Translate results back into a standardized output format
}
```

Finally, to manage the end-to-end execution of these linked models, an *orchestrator* function is employed. The orchestrator is responsible for iterating through the chain of assembled models, executing each one, and triggering the appropriate adaptors to pass the output of one model directly into the input of the next.

5 Case study: Multiple screening episodes

We reframe a previously published screening CEA model, built for the purposes of teaching and training in R (Lin et al. 2023). The example will demonstrate the different implementation approaches in a real-world case study. This is a CEA model that involves multiple screening sessions over a lifetime where, at the point of every screening session, individuals can be redistributed across health states in a MM based on their screening results (Afonso et al. 2023; Hazen 2002; Hollenberg 1984).

5.1 Data

The scenarios included are for five screening intervals of 2, 5, 10, 15, 20 years, and six total number of screening episodes 1, 2, 3, 4, 5, 10. Figure 2 shows the high-level diagram of the screening model from Lin et al. (2023). The Markov model and decision tree diagrams are shown in Figure 3 and Figure 4.

The data for the Markov model consist of the state transition probability matrix

$$P = \begin{bmatrix} 0.9 & 0.05 & 0 & 0 & 0 & 0.05 \\ 0 & 0.65 & 0.2 & 0 & 0.1 & 0.05 \\ 0 & 0 & 0.6 & 0.20 & 0.2 & 0 \\ 0 & 0 & 0 & 0.95 & 0 & 0.05 \\ 0 & 0 & 0 & 0 & 1.0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

and the unit state cost and health attributes are

$$\mathbf{c}^M = (0, 0, 0, 100, 0, 0)^\top, \quad \mathbf{u}^M = (1, 0.8, 0.5, 0.9, 0, 0)^\top$$

The M superscript indicates that these are for the Markov model. The data for the decision tree consist of the conditional transition probabilities along each branch

$$\begin{aligned} P(\text{FP} \mid \text{Healthy}) &= 0.1 \\ P(\text{TN} \mid \text{Healthy}) &= 0.9 \end{aligned}$$

$$\begin{aligned} P(\text{TP} \mid \text{Pre-clinical}) &= 0.7 \\ P(\text{FN} \mid \text{Pre-clinical}) &= 0.3 \end{aligned}$$

$$\begin{aligned} P(\text{TP} \mid \text{Disease}) &= 1.0 \\ P(\text{FN} \mid \text{Disease}) &= 0.0 \end{aligned}$$

and terminal state costs $c^D(x) = 200$ if TP or FP, and $c^D(x) = 50$ if TN or FN. The D superscript indicates that these are for the decision tree.

Finally, the initial distribution of individuals amongst the states of the Markov model is defined as

$$_0 = (0.80, 0.15, 0.05, 0, 0, 0)^\top$$

Notice that the simulation begins with a decision tree model screening round. The distribution of individuals at the start of a screening episode is represented with the branches from the root node of the decision tree which are therefore time-dependent.

6 R implementation

The analysis was carried-out in R version 4.6.0. All code is available on GitHub at https://github.com/n8thangreen/modularce. Because of limited space and to focus the exposition, we do not include function input checks (called *asserts*) and other additional code, which it is good practice to include.

The MM engine code is based on Green et al. (2023) and is considered a black-box for our purposes. It is sufficient to define it in terms of the *contract*, that is, what values it requires as inputs and what output we can expect to obtain from it. We will principally use base R functions in the examples. This is to minimise the

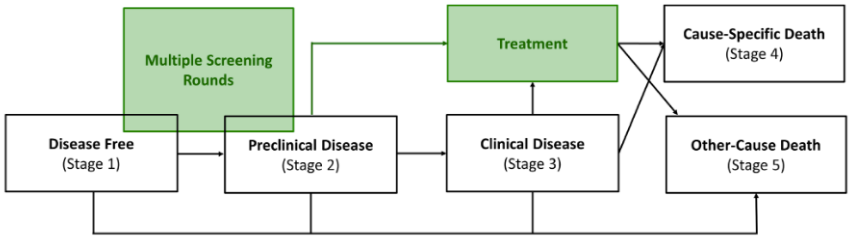


Figure 2: Screening model diagram taken from Lin et al. (2023). The white health states represent the natural history of disease. The two interventions, screening and treatment, are labelled in green. The screening intervention can only be implemented before the individual enters clinical health state (stage 3), and treatment can be received when patients are diagnosed from screening (stage 2) or clinically presented (stage 3).

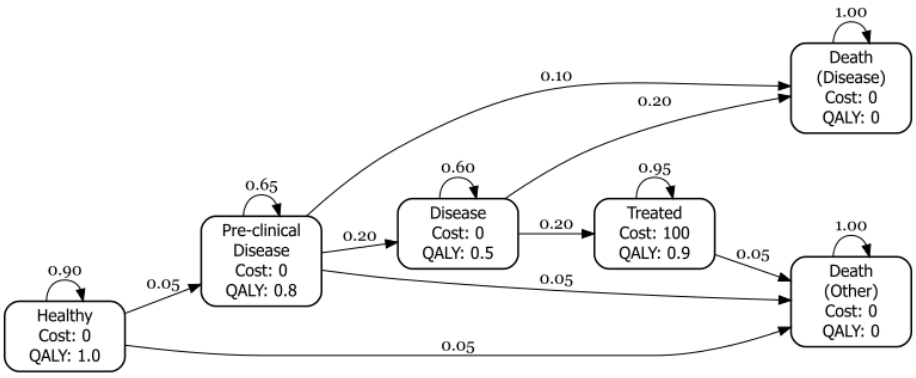


Figure 3: Markov model component diagram for the screening example.

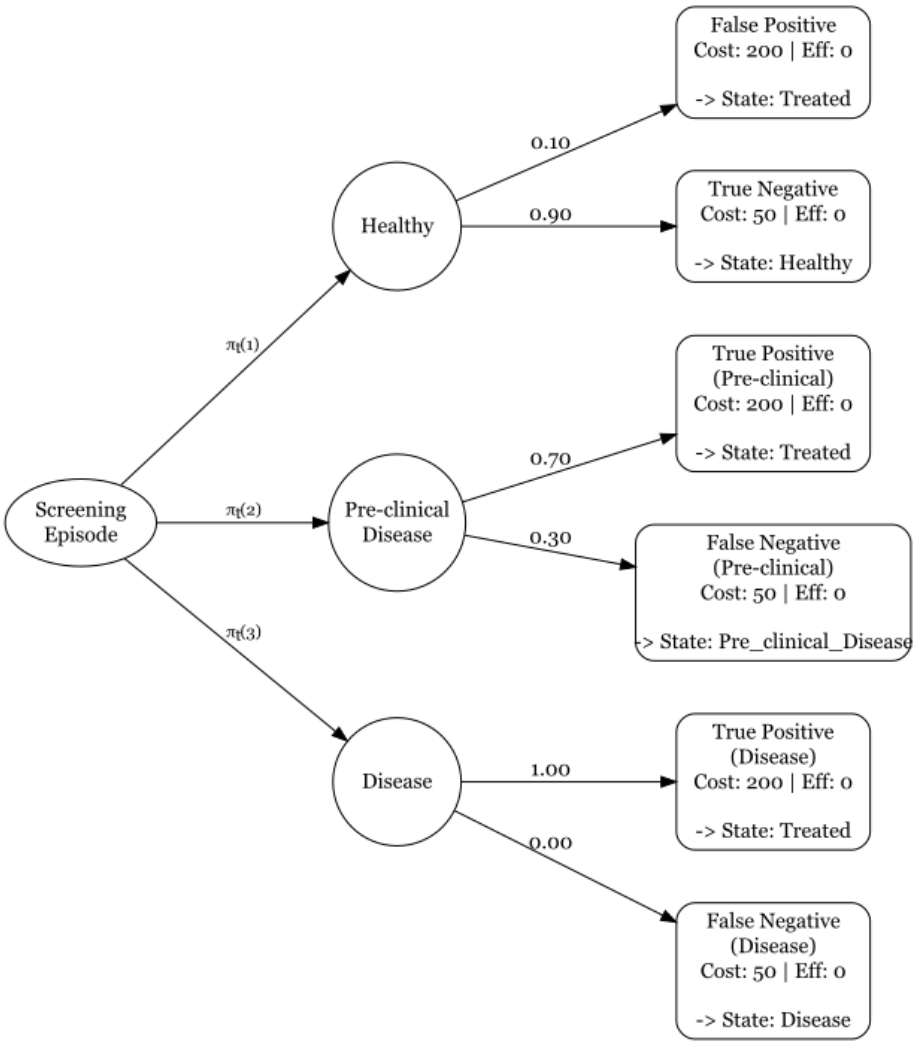


Figure 4: Decision tree model component diagram for the screening example.

number of dependencies and make the code more robust and reproducible. We consider the CHEERS checklist (Husereau et al. 2022) for the key components of an analysis which we need to take account of in our proposed framework. We do not discount costs or health to present values for this simple model, but in practice this is a core consideration.

6.1 S3 Framework

The framework leverages the S3 methodology to seamlessly transition patient cohorts. At each screening interval (2, 5, 10, 15, and 20 years), the DT calculates the immediate redistribution of individuals based on test outcomes. Instead of manual data hand-offs, the `update_model()` generic function captures these revised proportions and feeds them directly into the MM. This allows the MM to project long-term downstream costs and health outcomes continuously without breaking the computational pipeline.

The class constructor functions for a decision tree and a Markov model are:

```
DecisionTree <- function(data, disc_rate = 0.035) {
  structure(
    list(data = data,
         discount_rate = disc_rate,
         start_cycle   = 0),
    class = c("DecisionTree", "Model"))
}

MarkovModel <- function(trans_matrix,
                        costs, utils,
                        init_probs = NULL,
                        mapping = NULL,
                        n_cycles = 2,
                        disc_rate = 0.035) {
  structure(
    list(
      trans_matrix  = trans_matrix,
      cost_vector   = costs,
      util_vector   = utils,
      init_probs    = init_probs,
      mapping       = mapping,
      n_cycles      = n_cycles,
      discount_rate = disc_rate,
      start_cycle   = 0
    ),
    class = c("MarkovModel", "Model")
  )
}
```

The `structure()` function defines an object with additional attributes, in this case class.

We will also create a `CombinedModel()` constructor which will simply consist of a list of other models which have previously been created with their own class constructors as above. This structure will allow us to pass around a single object.

```
CombinedModel <- function(...) {
  structure(list(...),
            class = c("CombinedModel", "Model"))
}
```

We next define the generic functions, which are the polymorphic methods which will behave differently depending on what class of object is passed to them. This is 'boilerplate' R code which takes the same format every time.

```
run_model <- function(model, ...) {
  UseMethod("run_model")
}

update_model <- function(model, result) {
  UseMethod("update_model")
}
```

The class-specific `run_model()` methods for each of the submodels are defined next:

```
run_model.MarkovModel <- function(model, ...) {
  out <- markov_model(
    start_pop     = model$init_probs,
    trans_mat     = model$trans_matrix,
    cost_vec      = model$cost_vector,
    util_vec      = model$util_vector,
    n_cycles      = model$n_cycles,
    discount_rate = model$discount_rate,
    start_cycle   = model$start_cycle
  )

  list(
    expected_cost = out$expected_cost,
    expected_eff  = out$expected_eff,
    terminal      = out$final_state,
    trace         = out$trace,
    time_elapsed  = model$n_cycles
  )
}
```

```
run_model.DecisionTree <- function(model, ...) {
  path <- calculate_pathways(model$data)

  rate <- model$discount_rate
  cycle <- model$start_cycle

  discount_factor <- 1 / (1 + rate)^cycle

  list(
    expected_cost = sum(path$prob * path$cost) * discount_factor,
    expected_eff  = sum(path$prob * path$eff) * discount_factor,
    path_results  = path,
    time_elapsed  = 0
  )
}
```

As already stated, we assume the internal simulation engines of the decision tree, `calculate_pathways()`, and Markov model, `markov_model()`, as black boxes for our purposes. The full code is all available in the GitHub repo.

Next, we define the adaptor functions for each of the submodels. These are model-specific functions derived from the generic `update_model()`. In each case, results from the previous model in the pipeline are passed in and adapted appropriately to be input for the subsequent model, as defined by the class of the first argument.

```
update_model.MarkovModel <- function(model, prev_result) {
  if (!is.null(prev_result$path_results) && !is.null(model$mapping)) {
    all_nm <- colnames(model$trans_matrix)
    new_init <- setNames(rep(0, length(all_nm)), all_nm)

    paths <- prev_result$path_results
    paths$state <- model$mapping[paths$terminal_node]

    # Sum the probabilities for each state
    for (s in unique(paths$state)) {
      new_init[s] <- sum(paths$prob[paths$state == s])
    }

    model$init_probs <- new_init
  }
  model
}

update_model.DecisionTree <- function(model, prev_result) {
  if (!is.null(prev_result$terminal)) {
```

```
    end_states <- prev_result$terminal
    tree <- model$data
    root_idx <- which(
      tree$from == "root" &
        tree$to %in% names(end_states))
    tree$prob[root_idx] <- end_states[tree$to[root_idx]]
    model$data <- tree
  }
  model
}
```

The final key element is the orchestrator function. This higher-level function takes as first argument an object of class `CombinedModel` and steps through each of the component submodels by running and updating data for the appropriate submodel-specific methods of `run_model()` and `update_model()`.

```
run_model.CombinedModel <- function(model, ...) {
  results <- list()
  current_time <- 0  # initialise internal clock

  for (i in seq_along(model)) {
    curr_model <- model[[i]]

    # stamps model with current time
    curr_model$start_cycle <- current_time

    if (i > 1) {
      curr_model <- update_model(curr_model, results[[i - 1]])
    }

    res <- run_model(curr_model)
    results[[i]] <- res

    # advance clock using self-reported elapsed time
    current_time <- current_time + res$time_elapsed
  }
  results
}
```

6.2 Input data

Now that the function preparatory code is complete, the input data as defined in the Data section above are translated into R code. We define a simulation duration of 20 years and define the screening programme in terms of the time between screening and the number of screening episodes. The analysis will investigate all combinations of intervals and episode values.

```
# Define the grid of parameters
time_horizon <- 20
t_between_screens <- 1:10
num_screens <- 1:10
# t_between_screens <- c(2, 5, 10, 15, 20)
# num_screens <- c(1, 2, 3, 4, 5, 10)
# Set your new discount rate
discount_rate <- 0.035
```

The decision tree data are defined as a `tribble()` from the *tibble* package which allows us to write it in the same format that it will be printed for clearer code (**?@fig-code-input-dt**).

```
decision_tree <- tibble::tribble(
  ~from,                  ~to,                     ~prob, ~cost, ~eff,
  "root",                 "Healthy",               0.80,      0,    0,
  "root",                 "Pre_clinical_Disease",  0.15,      0,    0,  # Markov model
  "root",                 "Disease",               0.05,      0,    0,  # state
  "root",                 "Treated",               0.00,      0,    0,  # distribution
  "root",                 "Death_from_Disease",    0.00,      0,    0,
  "root",                 "Other_Cause_Death",     0.00,      0,    0,
  "Healthy",              "FP",                    0.10,    200,    0,  # False Positive
  "Healthy",              "TN",                    0.90,     50,    0,  # True Negative
  "Pre_clinical_Disease", "TP_Preclinical",        0.70,    200,    0,  # True Positive
  "Pre_clinical_Disease", "FN_Preclinical",        0.30,     50,    0,  # False Negative
  "Disease",              "TP_Disease",            1.00,    200,    0,  # True Positive
  "Disease",              "FN_Disease",            0.00,     50,    0   # False Negative
)
```

The Markov model transition probability matrix is defined in **?@fig-code-input-mm**.

```
states <- c(
  "Healthy", "Pre_clinical_Disease", "Disease", "Treated", "Death_from_Disease",
  "Other_Cause_Death")

trans_mat <- matrix(
  c(
    #    H     P     D     T     DD    OCD
    0.90, 0.05, 0.00, 0.00, 0.00, 0.05, # Healthy (H)
    0.00, 0.65, 0.20, 0.00, 0.10, 0.05, # Pre-clinical Disease (P)
    0.00, 0.00, 0.60, 0.20, 0.20, 0.00, # Clinical Disease (D)
    0.00, 0.00, 0.00, 0.95, 0.00, 0.05, # Treated (T)
    0.00, 0.00, 0.00, 0.00, 1.00, 0.00, # Death from Disease (DD) - Absorbing
    0.00, 0.00, 0.00, 0.00, 0.00, 1.00  # Other-Cause Death (OCD) - Absorbing
  ),
  nrow = 6, byrow = TRUE,
```

```
  dimnames = list(from = states,
                  to = states)
)
```

The state cost and health values are defined as follows:

```
cost_vec <- c(
  Healthy              = 0,
  Pre_clinical_Disease = 0,
  Disease              = 0,
  Treated              = 100,
  Death_from_Disease   = 0,
  Other_Cause_Death    = 0
)

util_vec <- c(
  Healthy              = 1.0,
  Pre_clinical_Disease = 0.8,
  Disease              = 0.5,
  Treated              = 0.9,
  Death_from_Disease   = 0.0,
  Other_Cause_Death    = 0.0
)

# Cohort distribution for natural history
init_pop <- c(
  Healthy              = 1.0,
  Pre_clinical_Disease = 0.0,
  Disease              = 0.0,
  Treated              = 0.0,
  Death_from_Disease   = 0.0,
  Other_Cause_Death    = 0.0
)
```

Finally, we need to define the mapping from the decision tree terminal nodes to the Markov model states:

```
mapping <- c(
  FP                 = "Treated",
  TN                 = "Healthy",
  TP_Preclinical     = "Treated",
  FN_Preclinical     = "Pre_clinical_Disease",
  TP_Disease         = "Treated",
  FN_Disease         = "Disease",
  Death_from_Disease = "Death_from_Disease",
  Other_Cause_Death  = "Other_Cause_Death",
  Treated            = "Treated"
```

```
)
```

6.3 Analysis

Having established the simulation code and input data, it is straightforward to run the analysis inside of a loop over the scenarios.

```
# Loop through scenarios and execute the unified framework

all_results <- list()
dt <- DecisionTree(decision_tree, disc_rate = discount_rate)

for (t_val in t_between_screens) {
  mm_scenario <-
    MarkovModel(trans_matrix = trans_mat,
                costs = cost_vec,
                utils = util_vec,
                init_probs = init_pop,
                mapping = mapping,
                n_cycles = t_val,
                disc_rate = discount_rate)

  for (n_scr in num_screens) {
    total_time <- n_scr * t_val

    # Filter out impossible combinations that
    # exceed the 20-year time horizon
    if (total_time > time_horizon) next

    submodels <- rep(list(dt, mm_scenario), n_scr)
    n_models <- length(submodels)

    t_remain <- time_horizon - total_time
    if (t_remain > 0) {
      submodels[[n_models]]$n_cycles <- t_val + t_remain
    }

    screening_model <- do.call(CombinedModel, args = submodels)

    results_scenario <- run_model(screening_model)

    result_label <- paste0("t_between_", t_val, "_screens_", n_scr)

    all_results[[result_label]] <- list(
      t_between_screens = t_val,
      num_screens       = n_scr,
```

```
      results           = results_scenario
    )
  }
}
```

6.4 Probabilistic sensitivity analysis

Probabilistic Sensitivity Analysis (PSA) is fundamentally streamlined under our unified framework. In a traditional basic approach, analysts must manually manage the extraction of DT outputs to feed into the MM for every single PSA iteration, which is both error-prone and computationally inefficient. By encapsulating the model within the `CombinedModel` class, a PSA simply requires applying a mapping function (such as `lapply()` in base R) over the probabilistic input distributions. The `run_model.CombinedModel()` function acts as the orchestrator, automatically handling the internal control flow and data passing for each iteration. This ensures that structural integrity is maintained across thousands of simulation runs without manual intervention.

6.5 Design choices

Even within the proposed architecture there are several subjective design decisions to make, which can depend on the specifics of the problem at hand. One design choice is where in the code to perform the linking of models. One option is to chain the models together when we create them or another option is to do this at the same time as running the model. There are pros and cons to each approach.

Another issue is to do with the generalisability of linking models. Should the code use the information about which type of model is being linked from (source) and linked to (target)? In an OOP framework this would require something called *double dispatch* which is not the original intention of S3 in R but can be manufactured using it and is implemented in packages such as *vctrs* (Wickham et al. 2026). Taking into account these considerations, in our case study example we have placed the linking logic within the model running function. We have also assumed that given a target model, we then know what the source model is. This is reasonable in our context and keeps the code clean for demonstration purposes. An alternative version of our code using the double dispatch approach employed by *vctrs* is also available in the supplementary material. The newer and less adopted S7 OOP system in R does natively support multiple dispatch which could elegantly solve the problem with a little more advanced code. We provide a version of the code in the supplementary material.

Our decision to utilize an S3-based orchestrator to link sequential model components aligns with established best practices in modern R software engineering. For instance, the *tidymodels* ecosystem (specifically the *workflows* and *stacks* packages) relies heavily on S3 generics to handle the sequential passing of data between preprocessing steps, individual models, and ensemble meta-models.

A Unified Markup Language (UML) diagram of the code structure is given in Appendix Figure 7. This shows how all of the functions are related to each other.

7 Results

Figure 5 illustrates the state occupancy over a 20-year time horizon for the 2-year and 10-year screening intervals. These plots demonstrate the expected dynamics: more frequent screening redistributes the population more rapidly into the "Treated" and "Disease" states.

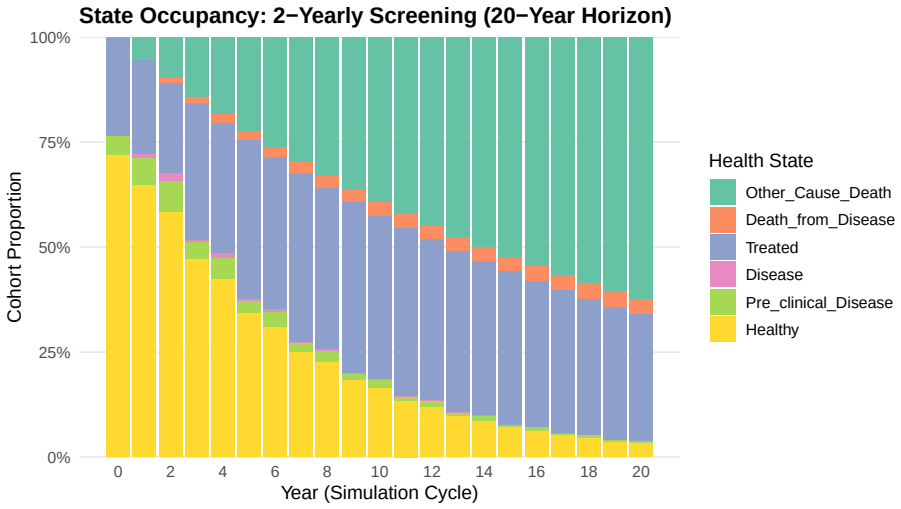
Figure 6 synthesizes the cost-effectiveness across the full grid of evaluated screening strategies, plotting total accumulated costs against total health outcomes. As expected from a structurally sound health economic evaluation, the results display clear efficiency trade-offs. Strategies utilizing highly frequent screening intervals (e.g., the 2-year interval curve) yield the highest overall health benefits by capturing clinical transitions earlier, but they do so at a significantly steeper marginal cost trajectory due to compounding screening and treatment expenses. Conversely, less frequent screening (e.g., the 15- and 20-year intervals) produces lower-cost outcomes with correspondingly reduced health gains.

Importantly, the smooth and mathematically predictable progression of these curves—scaling logically with the number of screening episodes—demonstrates that the declarative S3 framework functions exactly as intended. It confirms that the object-oriented architecture correctly accumulates costs and propagates complex cohort transitions between the DT and MM without the silent data-loss or misallocation errors common in manual imperative scripts.

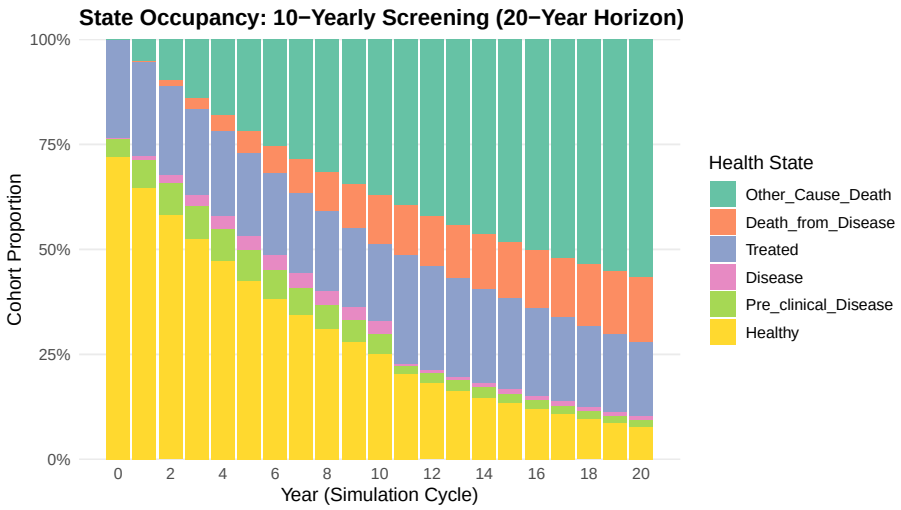
8 Changes in Model and Data

8.0.1 Submodel substitution

```
# constructor
PartSAModel <- function(surv_pfs, surv_os, cost_vector,
                        qaly_vector, mapping = NA, time_horizon = 10, ...) {
  structure(
    list(
      surv_pfs     = surv_pfs,     # parametric survival curves
      surv_os      = surv_os,
      cost_vector  = cost_vector,
      qaly_vector  = qaly_vector,
      mapping      = mapping,      # links tree terminal nodes to baseline survival risk
      time_horizon = time_horizon
    ),
    class = c("PartSAModel", "Model")
  )
```



(a) 2-yearly screening (10 rounds, 20-year horizon)



(b) 10-yearly screening (2 rounds, 20-year horizon)

Figure 5: State occupancy plots comparing 2-year and 10-year screening intervals.

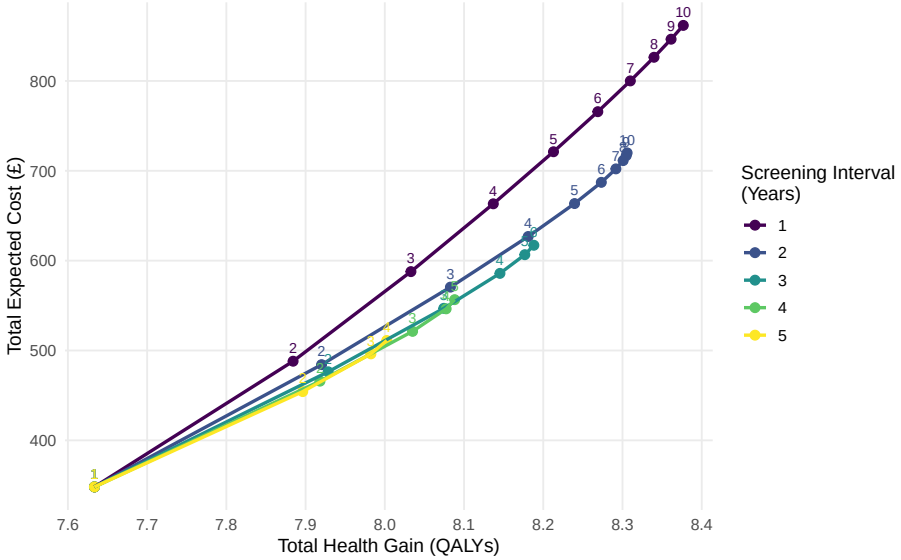


Figure 6: Cost-effectiveness plot for screening intervals of 1 to 5 years, with all screening episodes numbered.

```
}
```

S3 methods

```
#' S3 update method: Ingests Decision Tree outputs into PartSA baseline parameters
#' @export
update_model.PartSAModel <- function(model, result) {
  # Map tree probabilities to adjust scale or
  # baseline hazard for each treatment arm
  model$baseline_weights <- map_decision_to_partsa(result, model$mapping)
  model
}
```

```
#' S3 runner method: Solves the area under the curves
#' @export
run_model.PartSAModel <- function(model) {
  # Evaluate partitioned survival areas (PFS, OS - PFS, 1 - OS)
  out <- run_partsa_engine(
    pfs_fit    = model$surv_pfs,
    os_fit     = model$surv_os,
    weights    = model$baseline_weights,
    horizon    = model$time_horizon
  )
```

```
  structure(
    list(
      expected_cost = sum(out$state_costs * model$cost_vector),
      expected_eff  = sum(out$state_qalys * model$qaly_vector),
      survival_mesh = out$mesh
    ),
    class = c("output", class(model))
  )
}
```

Analysis

```
# Swap Markov for PartSA without touching runner code:
model_c <- PartSAModel(surv_pfs = fit_pfs, surv_os = fit_os,
                       cost_vector = cost_vec, qaly_vector = util_vec,
                       mapping = map_ps)
chain_partsa <- CombinedModel(model_a, model_c)

# Identical orchestration call
res_partsa <- run_model(chain_partsa)
```

8.0.2 Structural Adaptations via S3 Class Specialisation

When a jurisdiction mandates a structural change—such as an alternative long-term extrapolation method or an additional post-progression therapy branch—you use class derivation (e.g., `c("MarkovModel_PBAC", "MarkovModel", "Model")`).

```
# PBAC structural variant: models continuous PBS dynamic market-share erosion
MarkovModel_PBAC <- function(trans_matrix, costs, utils,
                             erosion_rate = 0.05, mapping = NULL, ...) {
  mod <- MarkovModel(trans_matrix = trans_matrix, costs = costs,
                     utils = utils, mapping = mapping, ...)
  mod$erosion_rate <- erosion_rate
  class(mod) <- c("MarkovModel_PBAC", class(mod))
  mod
}

# Specific S3 method dispatch: PBAC applies market-share price cuts over cycles
run_model.MarkovModel_PBAC <- function(model) {
  # Apply country-specific dynamic erosion adjustment before running
  model$cost_vector <- apply_pbs_erosion(model$cost_vector, model$erosion_rate)

  # Delegate core execution to base Markov runner
  NextMethod("run_model")
}
```

`NextMethod()` delegates the heavy matrix operations directly to the verified global engine (`run_model.MarkovModel`). The central orchestrator `run_model.CombinedModel` runs without touching a line of code.

9 Discussion

Traditional health economic models often bundle clinical pathways, cost inputs, and final calculations into a single, rigid script or spreadsheet, making them fragile and difficult to update. In contrast, our framework applies established software engineering standards—specifically the SOLID principles—to create a much more robust and adaptable system. By ensuring that each component has only one job (Single Responsibility), such as completely separating the clinical simulation from the economic payoffs, we drastically reduce the risk of errors when updating parameters. Furthermore, the architecture is designed so that new components, like a custom jurisdiction profile or a new sub-model, can be plugged into the pipeline without altering or breaking the core mathematical engine (Open-Closed Principle). Because all sub-models communicate through a standardised data contract, researchers can swap a Markov model for a different structure with confidence that the downstream calculations will still function perfectly (Liskov Substitution). Ultimately, this approach moves health economic modelling away from bespoke, tangled code and towards a transparent, modular infrastructure where each piece can be independently tested and trusted.

10 Conclusion

We have demonstrated that adopting a declarative, object-oriented programming approach using S3 in R provides a robust and scalable solution for multi-component cost-effectiveness models. By abstracting the control flow and automating data transitions between DTs and MMs, this framework directly addresses the testability, reproducibility, and efficiency issues prevalent in legacy imperative scripts. Although moving away from traditional step-by-step coding requires a paradigm shift for many health economists, the resulting unified architecture aligns health technology assessment with modern software engineering best practices. Ultimately, this improves the transparency, flexibility, and reliability of the models used to inform critical healthcare decisions.

Acknowledgements

xxx

References

Afonso, D, H Davies, B Avey, and S Mealing. 2023. “MSR8 Nesting a Decision Tree into a Markov Model Structure: An Alternative Way of Tracking

Movement in a Diagnostic Model." *Value Heal.* 26 (12): S394. https://doi.org/10.1016/j.jval.2023.09.2067.

Alarid-Escudero, Fernando, Eline M. Krijkamp, Eva A. Enns, et al. 2019. "A Framework for Decision-Analytic Modeling in R: Implementing Tertiary Prevention of Colorectal Cancer with a Markov Model." *Medical Decision Making* 39 (2): 175–84. https://doi.org/10.1177/0272989X18814775.

Briggs, Andrew, Karl Claxton, and Mark Sculpher. 2006. *Decision Modelling for Health Economic Evaluation*. Oxford University Press.

Drummond, Michael F., Mark J. Sculpher, Karl Claxton, Greg L. Stoddart, and George W. Torrance. 2015. *Methods for the Economic Evaluation of Health Care Programmes*. 4th ed. Oxford University Press.

Eddy, David M., William Hollingworth, J. Jaime Caro, Robert Itzler, and Mark S. Townsend. 2012. "Model Transparency and Validation: A Report of the ISPOR-SMDM Modeling Good Research Practices Task Force–7." *Medical Decision Making* 32 (5): 733–43. https://doi.org/10.1177/0272989X12454504.

Green, Nathan, Felicity Lamrock, Niamh Naylor, et al. 2023. "Health Economic Evaluation Using Markov Models in r for Microsoft Excel Users: A Tutorial." *PharmacoEconomics* 41 (1): 5–19. https://doi.org/10.1007/s40273-022-01199-7.

Hazen, Gordon B. 2002. "Stochastic Trees and the StoTree Modeling Environment: Models and Software for Medical Decision Analysis." *J. Med. Syst.* 26 (5).

Hollenberg, JP. 1984. "Markov Cycle Trees-a New Representation for Complex Markov-Processes." *Medical Decision Making* 4: 529–29.

Husereau, Don, Michael Drummond, Federico Augustovski, et al. 2022. "Consolidated Health Economic Evaluation Reporting Standards 2022 (CHEERS 2022) Statement: Updated Reporting Guidance for Health Economic Evaluations." *Value in Health* 25 (January): 3–9. https://doi.org/10.1016/j.jval.2021.11.1349.

Jalal, Hawre, Petros Pechlivanoglou, Eline Krijkamp, Fernando Alarid-Escudero, Eva Enns, and M. G. Myriam Hunink. 2017. "An Overview of R in Health Decision Sciences." *Medical Decision Making* 37 (7): 735–46. https://doi.org/10.1177/0272989X16679605.

Kent, Seamus, Frauke Becker, Talitha Feenstra, et al. 2019. "The Challenge of Transparency and Validation in Health Economic Decision Modelling: A View from Mount Hood." *PharmacoEconomics* 37 (11): 1305–12. https:

//doi.org/10.1007/s40273-019-00825-1.

Krijkamp, Eline M., Fernando Alarid-Escudero, Eva A. Enns, Hawre J. Jalal, M. G. Myriam Hunink, and Petros Pechlivanoglou. 2018. "A Checklist for Generating, Reporting and Modelling Decision-Analytic Models in R." *Medical Decision Making* 38 (8): 1000–1012. https://doi.org/10.1177/0272989X18811838.

Kuhn, Max, and Hadley Wickham. 2020. *Tidymodels: A Collection of Packages for Modeling and Machine Learning Using Tidyverse Principles.* https://www.tidymodels.org.

Lin, Y. S., J. F. O'Mahony, and J. van Rosmalen. 2023. "A Simple Cost-Effectiveness Model of Screening: An Open-Source Teaching and Research Tool Coded in R." *PharmacoEconomics - Open* 7 (4): 507–23. https://doi.org/10.1007/s41669-023-00414-1.

R Core Team. 2025. *R: A Language and Environment for Statistical Computing.* R Foundation for Statistical Computing. https://www.R-project.org/.

Sandercock, Peter, Eivind Berge, Martin Dennis, et al. 2004. "Cost-Effectiveness of Thrombolysis with Recombinant Tissue-Type Plasminogen Activator for Acute Ischemic Stroke: Assessed by a Model Based on UK NHS Costs." *Stroke* 35 (6): 1490–97. https://doi.org/10.1161/01.STR.0000128706.77784.b8.

Wickham, Hadley, Lionel Henry, and Davis Vaughan. 2026. *Vctrs: Vector Helpers.* https://vctrs.r-lib.org/.

Wu, Eric Q., Zheng-Yi Zhou, Jipan Xie, Cinzia Metallo, and Praveen Thokala. 2019. "Transparency in Health Economic Modeling: Options, Issues and Potential Solutions." *PharmacoEconomics* 37 (11): 1349–54. https://doi.org/10.1007/s40273-019-00842-0.

Appendix

10.1 UML

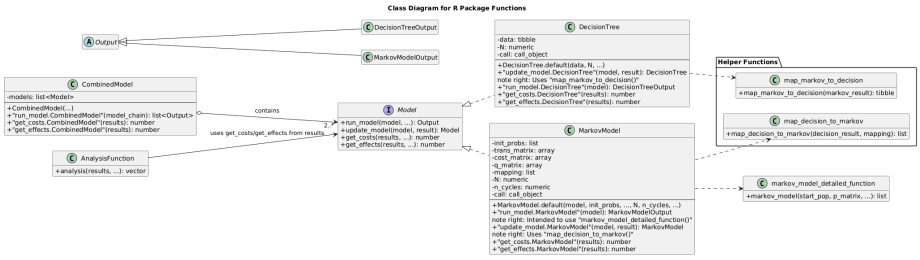


Figure 7: UML diagram for the decision tree - Markov model code. Inheritance / Generalization is a solid line with a large hollow (unfilled) triangle arrowhead pointing to the parent class in our case the decision tree and Markov model output; Implementation is a dashed line with a large hollow (unfilled) triangle arrowhead pointing to the interface, in our case the specific type of models; Dependency is a dashed line with an open arrowhead, the function on the right hand side of the diagram. The line with the unfilled diamond represents aggregation and "2..*" indicates that there are 2 or more models in `CombinedModel`.